Answerspot — GitHub Actions CI/CD

Three workflows at `.github/workflows/`, matching DESIGN_SPEC §9.

```
on PR      → ci.yml       lint · typecheck · test · build · trivy · docker build
(no push)
on main    → deploy.yml   above + push images → deploy staging → smoke tests
on tag v*  → release.yml  manual approval → promote to prod → post-deploy
healthcheck
```

`.github/workflows/ci.yml`

yaml

```yaml
name: CI

on:
  pull_request:
    branches: [main]

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

jobs:
  node:
    name: API + Web (lint · typecheck · test · build)
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16-alpine
        env:
          POSTGRES_USER: answerspot
          POSTGRES_PASSWORD: answerspot
          POSTGRES_DB: answerspot_test
        ports: ["5432:5432"]
        options: >-
          --health-cmd "pg_isready -U answerspot"
          --health-interval 5s --health-timeout 5s --health-retries 5
    env:
      DATABASE_URL: postgres://answerspot:answerspot@localhost:5432/answerspot_test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - name: Prisma generate + migrate (test DB)
        run: |
          npx prisma generate --schema apps/api/prisma/schema.prisma
          npx prisma migrate deploy --schema apps/api/prisma/schema.prisma
      - run: npm run test
      - run: npm run build
```

```yaml
  python:
    name: Workers (ruff · mypy · pytest)
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: apps/workers
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: mypy answerspot_workers
      - run: pytest -q

  security:
    name: Security scan (Trivy)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Trivy filesystem scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          scan-ref: .
          severity: CRITICAL,HIGH
          exit-code: "1"
          ignore-unfixed: true

  docker-build:
    name: Docker build (no push)
    needs: [node, python]
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [web, api, worker]
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - name: Build ${{ matrix.service }} image
        uses: docker/build-push-action@v6
        with:
```

```
          context: .
          file: infra/docker/${{ matrix.service }}.Dockerfile
          push: false
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

.github/workflows/deploy.yml

yaml

```yaml
name: Deploy (staging)

on:
  push:
    branches: [main]

concurrency:
  group: deploy-staging
  cancel-in-progress: false

permissions:
  contents: read
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE_PREFIX: ghcr.io/${{ github.repository }}

jobs:
  build-push:
    name: Build & push images
    runs-on: ubuntu-latest
    outputs:
      tag: ${{ steps.meta.outputs.tag }}
    strategy:
      matrix:
        service: [web, api, worker]
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - id: meta
        run: echo "tag=sha-${GITHUB_SHA::12}" >> "$GITHUB_OUTPUT"
      - name: Build & push ${{ matrix.service }}
        uses: docker/build-push-action@v6
        with:
          context: .
          file: infra/docker/${{ matrix.service }}.Dockerfile
```

```
        push: true
        tags: |
          ${{ env.IMAGE_PREFIX }}/${{ matrix.service }}:${{ steps.meta.outputs.tag
          ${{ env.IMAGE_PREFIX }}/${{ matrix.service }}:staging
        cache-from: type=gha
        cache-to: type=gha,mode=max

  deploy-staging:
    name: Helm deploy → staging
    needs: build-push
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - name: Configure kubeconfig
        run: |
          mkdir -p ~/.kube
          echo "${{ secrets.KUBECONFIG_STAGING }}" | base64 -d > ~/.kube/config
      - uses: azure/setup-helm@v4
      - name: Helm upgrade
        run: |
          helm upgrade --install answerspot infra/helm \
            --namespace answerspot-staging --create-namespace \
            --set image.tag=${{ needs.build-push.outputs.tag }} \
            --values infra/helm/values.staging.yaml \
            --wait --timeout 5m
      - name: Run DB migrations (Job)
        run: |
          kubectl -n answerspot-staging create job --from=cronjob/prisma-migrate \
            migrate-${{ needs.build-push.outputs.tag }} || true
          kubectl -n answerspot-staging wait --for=condition=complete \
            job/migrate-${{ needs.build-push.outputs.tag }} --timeout=180s

  smoke-test:
    name: Smoke tests
    needs: deploy-staging
    runs-on: ubuntu-latest
    steps:
      - name: Health check
        run: |
          for i in $(seq 1 10); do
            if curl -fsS "${{ vars.STAGING_API_URL }}/health"; then exit 0; fi
            sleep 10
```

```
            done
            echo "Staging health check failed"; exit 1
```

.github/workflows/release.yml

yaml

```yaml
name: Release (production)

on:
  push:
    tags: ["v*"]

permissions:
  contents: read
  packages: write

env:
  REGISTRY: ghcr.io
  IMAGE_PREFIX: ghcr.io/${{ github.repository }}

jobs:
  promote:
    name: Retag staging images → release tag
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: [web, api, worker]
    steps:
      - name: Log in to GHCR
        uses: docker/login-action@v3
        with:
          registry: ${{ env.REGISTRY }}
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Retag :staging → ${{ github.ref_name }}
        run: |
          docker buildx imagetools create \
            --tag ${{ env.IMAGE_PREFIX }}/${{ matrix.service }}:${{ github.ref_name }}
            --tag ${{ env.IMAGE_PREFIX }}/${{ matrix.service }}:prod \
            ${{ env.IMAGE_PREFIX }}/${{ matrix.service }}:staging

  deploy-prod:
    name: Helm deploy → production
    needs: promote
    runs-on: ubuntu-latest
    environment: production   # requires manual approval (configure in repo settings)
    steps:
      - uses: actions/checkout@v4
      - name: Configure kubeconfig
```

```
      run: |
        mkdir -p ~/.kube
        echo "${{ secrets.KUBECONFIG_PROD }}" | base64 -d > ~/.kube/config
    - uses: azure/setup-helm@v4
    - name: Run DB migrations (Job)
      run: |
        kubectl -n answerspot-prod create job --from=cronjob/prisma-migrate \
          migrate-${{ github.ref_name }} || true
        kubectl -n answerspot-prod wait --for=condition=complete \
          job/migrate-${{ github.ref_name }} --timeout=300s
    - name: Helm upgrade
      run: |
        helm upgrade --install answerspot infra/helm \
          --namespace answerspot-prod --create-namespace \
          --set image.tag=${{ github.ref_name }} \
          --values infra/helm/values.prod.yaml \
          --wait --timeout 10m
    - name: Post-deploy health check
      run: |
        for i in $(seq 1 15); do
          if curl -fsS "${{ vars.PROD_API_URL }}/health"; then exit 0; fi
          sleep 10
        done
        echo "Production health check failed — investigate before rollback"; exit 1

  rollback-on-failure:
    name: Rollback on failed deploy
    needs: deploy-prod
    if: failure()
    runs-on: ubuntu-latest
    environment: production
    steps:
      - name: Configure kubeconfig
        run: |
          mkdir -p ~/.kube
          echo "${{ secrets.KUBECONFIG_PROD }}" | base64 -d > ~/.kube/config
      - uses: azure/setup-helm@v4
      - run: helm rollback answerspot --namespace answerspot-prod --wait
```

Required repo configuration

Secrets (Settings → Secrets and variables → Actions):

- `KUBECONFIG_STAGING`, `KUBECONFIG_PROD` — base64-encoded kubeconfigs
- `GITHUB_TOKEN` — auto-provided for GHCR

Variables:

- `STAGING_API_URL`, `PROD_API_URL` — for health checks

Environments (Settings → Environments):

- `staging` — auto-deploy on main
- `production` — **require reviewers** (the manual approval gate from the spec)

Prerequisites the eng team must provide:

- `infra/docker/{web,api,worker}.Dockerfile`
- `infra/helm/` chart + `values.staging.yaml` / `values.prod.yaml`
- A `prisma-migrate` CronJob (suspended; used as a template for migration Jobs)
- A `/health` endpoint on the API
- Root `package.json` scripts: `lint`, `typecheck`, `test`, `build`